

Управление учебным стендом BALUS

Дата: 2026.01.24

Автор: Шендрик Павел Дмитриевич

Оглавление

Оглавление	1
Учебный стенд BALUS	1
ПИД-регулятор	2
Практическая реализация системы с ПИД-регулятором	4
Схема учебного стенда BALUS	6
Описание программы	7
Работа программы	9
Список литературы.....	11

Учебный стенд BALUS

BALUS (от англ. *BAL*ancing, *Ultrasonic Sensor*; балансирующий с ультразвуковым датчиком) – учебный стенд (рис. 1), демонстрирующий работу ПИД-регулятора, представляет собой вагонетку, постоянно балансирующую на подвижной поверхности, ультразвуковой датчик измерения расстояния *HC-SR04* [1], сервопривода *MG996R* [2], платы *STM32F103 Blue Pill* и блока питания.

Плата постоянно получает данные от ультразвукового датчика и корректирует поверхность под вагонеткой таким образом, чтобы вагонетка всегда оставалась на одном, заранее заданном, месте.

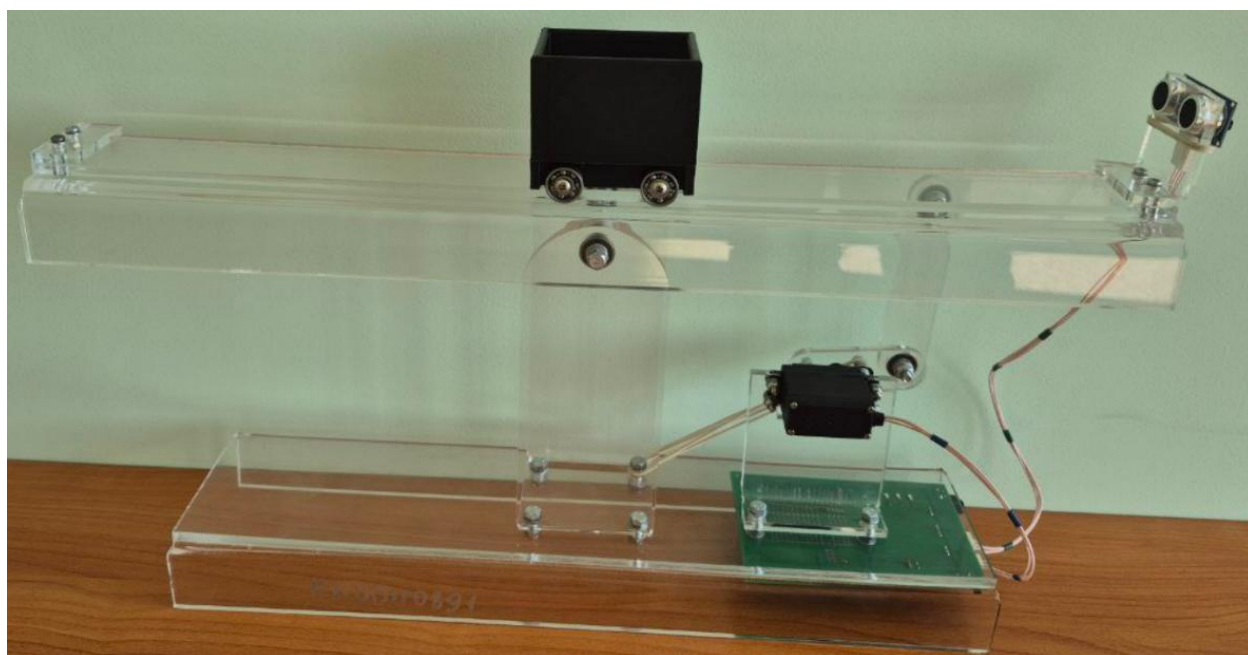


Рис. 1 Учебный стенд *BALUS*

ПИД-регулятор

Регулятор в курсе ТАУ – такое устройство, которое решает задачу управления какими-либо механизмами (двигателями, нагревателями, освещением и т.п.), а ПИДрегулятор – это разновидность таких регуляторов. Он является одним из самых универсальных, простых в понимании и настройке (его можно настроить буквально методом перебора).

Чтобы понять принцип работы ПИД-регулятора, представим трёх человек, которые вместе сидят на одном велосипеде (рис. 2):



Рис. 2 Три человека на велосипеде

- Первый человек (П – пропорциональная составляющая) – смотрит прямо вперёд и сразу реагирует, если велосипед уходит в сторону. Он пытается быстро исправить положение, чтобы велосипед ехал прямо;
- Второй человек (И – интегральная составляющая) – постоянно оглядывается назад и запоминает все прошлые отклонения. Если велосипед долго уходит в сторону, он постепенно прикладывает усилия, чтобы вернуть его в правильное положение;
- Третий человек (Д – дифференциальная составляющая) – также смотрит вперёд и предугадывает, как будет развиваться ситуация. Если велосипед наклоняется или уходит в сторону, он быстро реагирует, пытаясь предотвратить сильное отклонение.

Вместе они работают как команда: один сразу исправляет отклонение, другой учитывает прошлое и помогает не допустить долгого отклонения, а третий предугадывает будущее и помогает быстро реагировать на изменения. Так велосипед едет ровно и стабильно. Но при неправильной групповой работе велосипед может упасть.

Таким образом в ПИД-регуляторе:

- Пропорциональная часть (П): Реагирует сразу на текущую ошибку. Чем больше ошибка – тем сильнее система пытается её исправить прямо сейчас;
- Интегральная часть (И): Запоминает все прошлые ошибки и помогает устранить постоянные смещения или небольшие отклонения, которые не исчезают со временем;
- Дифференциальная часть (Д): Предсказывает будущее изменение ошибки по скорости её роста или уменьшения. Помогает сгладить реакцию и избежать рывков в системе.

Правильно настроенный ПИД-регулятор помогает системе быть быстрой, точной и стабильной за счёт балансировки трёх составляющих: пропорциональной (быстрая реакция), интегральной (устранение постоянных ошибок) и дифференциальной (предсказание будущих изменений).

Практическая реализация системы с ПИД-регулятором

На вход системы с ПИД-регулятором (рис. 3) подаётся желаемое расстояние до вагонетки *target*, которое после работы ПИД-регулятора задаёт управляющее воздействие на сервопривод *servo*, чтобы переместить вагонетку в нужную позицию. С помощью ультразвукового датчика измерения расстояния измеряется фактическое положение вагонетки *sensor*, которое используется в обратной связи для вычисления ошибки регулирования. Система управления постоянно (не чаще, чем раз в 10 мс) следит за изменением положения вагонетки и производит вычисление управляющего воздействия.

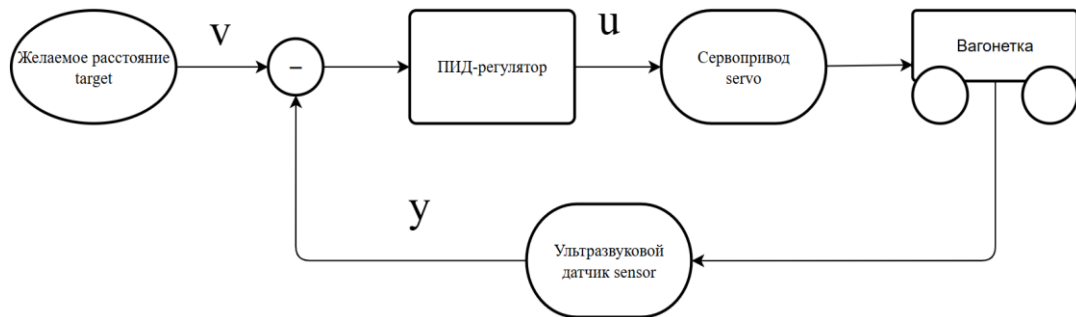


Рис. 3 Структурная схема системы

Существует множество программных реализаций [3], отличающихся способами интегрирования и дифференцирования, наличием дополнительных коэффициентов и переменных, и т.д. В программе (рис. 4) используется модификация ПИД-регулятора с дифференциальным фильтром второго порядка¹. Результатом работы регулятора является управление *pid_u* (рис. 4), которое получается путём сложения всех компонент² регулятора.

```
// Ошибка регулирования: желаемое значение v - действительное y
pid_error = target_distance - distance_sensor_result_mm();
pid_dt = pid_period / 1000.0; // Период дискретизации регулятора, с
// Дифференцирующий фильтр 2-го порядка
filter_d2y = pid_error / (filter_T * filter_T) - 2 * filter_d * \
            filter_dy / filter_T - filter_y / (filter_T * filter_T);
filter_dy += filter_d2y * pid_dt;
filter_y += filter_dy * pid_dt;
pid_p = pid_error; // Пропорциональная часть
pid_i += pid_error * pid_dt; // Интегральная часть
pid_d = filter_dy; // Дифференциальная часть
pid_u = k_p * pid_p + k_i * pid_i + k_d * pid_d; // Управление u
```

Рис. 4 Программная реализация ПИД-регулятора

¹ При практической реализации системы с ПИД-регулятором возникает множество трудностей, одной из которых является усиление помех дифференциальной составляющей. Поэтому для фильтрации помех и оценки производной добавляется дифференцирующий фильтр [4].

² Каждая компонента регулятора (П, И или Д) представляется как произведение коэффициента усиления (k_p , k_i или k_d), который может изменяться пользователем, и соответствующей составляющей (пропорциональной, интегральной или дифференциальной), которая изменяется программно в регуляторе.

Для взаимодействия с сервоприводом управление нормализуется путём получения разности между начальным положением сервопривода *servo_initial*³ и управлением *pid_u* (рис. 5). Нормализовать управление с помощью разности необходимо для компенсации инвертированного направления вращения сервопривода, которое возникает из-за текущего расположения и ориентации сервопривода.

```
// Управляющее воздействие на сервопривод  
TIM2->CCR4 = servo_initial - pid_u;
```

Рис. 5 Управляющее воздействие на сервопривод

³Сервопривод MG996R имеет угол поворота от 0° до 180° поэтому, для поворота сервопривода как по, так и против часовой стрелки, необходимо задавать начальное положение сервопривода *servo_initial*, чтобы установить его в положение 90°.

Схема учебного стенда BALUS

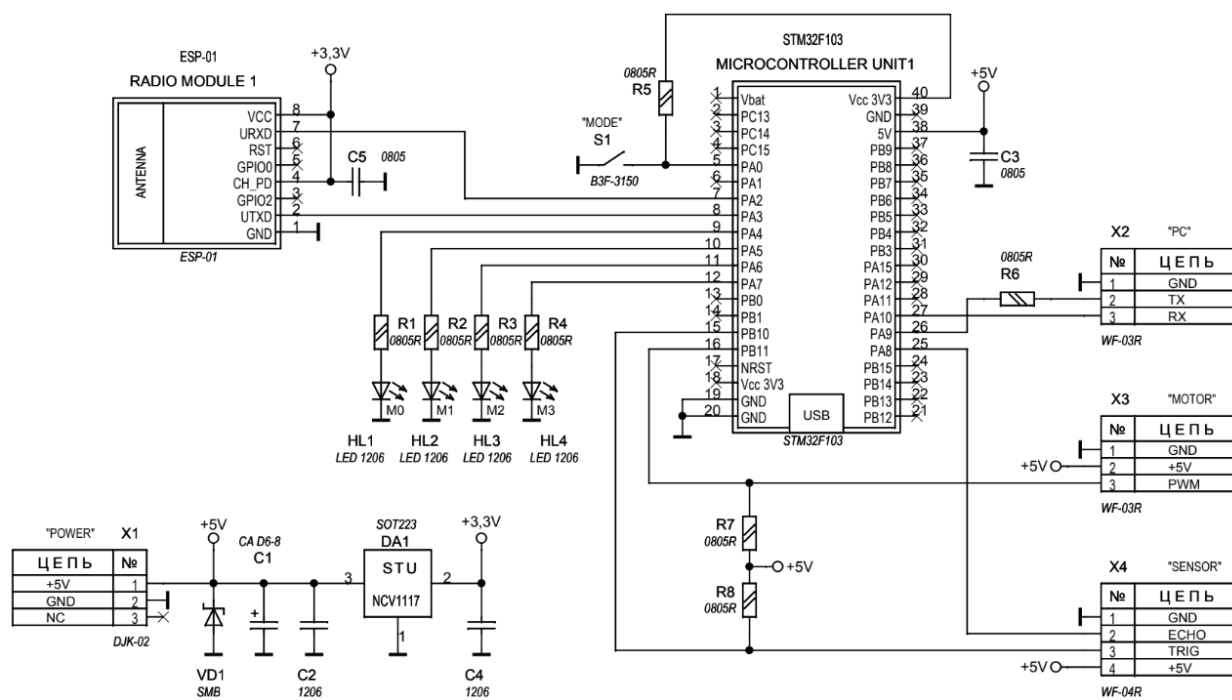


Рис. 6 Схема учебного стенда BALUS

Описание программы

Программной основой учебного стенда *BALUS* является простейшая операционная система (далее «ОС») – *PicOS*, обеспечивающая минимальный набор базовых функций. Программа состоит из конечных автоматов [5, 6], постоянно взаимодействующих друг с другом. Главной задачей ОС является общение между системой и пользователем: обработка полученных строк, ответ на команды, вывод и изменение переменных.

В главном цикле *while* (рис. 7) находятся вставки *#include* с содержимым всех автоматов. Такой способ представления информации используется для экономии места, улучшения читаемости главного цикла и ускорения работы системы – во время компиляции программы, вставка заменяется на соответствующий файл, что приравнивается к копированию всего содержимого автомата в *while*, а использование функций, вместо вставок сказывается на быстродействии (т.к. используются вызовы и возвраты из функций), что очень критично для встраиваемых систем. Все автоматы находятся в папке «Tasks», они содержат собственное краткое описание и комментарии к коду.

```
// ===== ГЛАВНЫЙ ЦИКЛ =====  
while (1) {  
    #include "Tasks/usart1_send.c" // Отправка данных по USART1  
    #include "Tasks/usart1_rcv.c"  // Принятие строки по USART1  
    #include "Tasks/picos.c"       // Реализация PicOS  
    #include "Tasks/var_get.c"     // Вывод значений переменных по USART1  
    #include "Tasks/var_set.c"     // Изменение значений переменных  
    #include "Tasks/distance_sensor.c" // Измерение расстояния до объекта  
    #include "Tasks/pid.c"        // ПИД-регулятор  
    #include "Tasks/button.c"     // Обработка нажатия кнопки  
}
```

Рис. 7 Главный цикл программы

Все автоматы можно условно разделить на системные, главные и пользовательские:

- Системные автоматы (*usart1_rcv*, *usart1_send*, *var_get*, *var_set*) выполняют функции операционной системы: обработку ввода/вывода, управление переменными.
- Главные автоматы (*picos*, *pid*) опрашивают и управляют работой других автоматов, а также анализируют полученные команды.
- Пользовательские автоматы (*distance_sensor*, *button*) реализуют конкретную функцию управления или опрашивания.

Таким образом шестой автомат *distance_sensor* нужен для измерения расстояния, с помощью ультразвукового датчика *HC-SR04*. Результат постоянных измерений расстояния можно узнать командой «sensor». Седьмой *pid* – обеспечивает работу ПИД регулятора. Пропорциональный *k_p*, интегральный *k_i* и дифференцирующий *k_d* коэффициенты регулятора, также как и коэффициенты дифференцирующего фильтра *filter_d*, *filter_T* можно изменять командами «k_p ...», «f_d ...», «f_t ...» и т.п. А восьмой автомат *button* нужен для обработки нажатия кнопки, чтобы изменять переменную «target», отвечающую за желаемое расстояние до вагонетки, без использования

компьютера (автономный режим). Все команды (*sensor*, *k_p* и т.п.) изменяют или запрашивают значения переменных, которые и отслеживаются в программе.

Все переменные, которые можно изменить или запросить командой, описываются в таблице переменных *var_table[]* (рис. 8), каждый элемент таблицы представляет собой структуру из четырёх элементов: Отображаемое имя переменной *name[12]*, краткое описание переменной *description[32]*, тип переменной *type* (*uint16_t* или *float*) и указатель на саму переменную **ptr*. Таким образом таблица из переменных будет выглядеть так:

```
// ===== Инициализация таблицы переменных =====
var_table_t var_table[] = {
    {"version", "Version of the firmware", TYPE_UINT16_T, &fm_version},
    {"sensor", "HC-SR04 reading, mm", TYPE_UINT16_T, &distance_sensor_value_mm},
    {"error", "regulatory error", TYPE_FLOAT, &pid_error},
    {"k_p", "Value of the P coeff", TYPE_FLOAT, &k_p},
    {"k_i", "Value of the I coeff", TYPE_FLOAT, &k_i},
    {"k_d", "Value of the D coeff", TYPE_FLOAT, &k_d},
    {"f_d", "Value of the d filter coeff", TYPE_FLOAT, &filter_d},
    {"f_t", "Value of the T filter coeff", TYPE_FLOAT, &filter_T},
    {"dt", "Sampling interval, ms", TYPE_UINT16_T, &pid_period},
    {"target", "Target cart distance, mm", TYPE_UINT16_T, &target_distance},
    {"servo", "Control signal, 1500..4000", TYPE_UINT16_T, &servo_drive_switch}
};
```

Рис. 8 Таблица переменных *var_table[]*

Где, *version* – версия программы (равна 34, используется для преобразования в 3.4);

sensor – расстояние, измеряемое датчиком *HC-SR04*, в мм;

error – ошибка регулирования;

k_p – значение пропорционального коэффициента ПИД регулятора;

k_i – значение интегрального коэффициента ПИД регулятора;

k_d – значение дифференциального коэффициента ПИД регулятора;

f_d – значение *d* коэффициента фильтра;

f_t – значение *T* коэффициента фильтра;

dt – период дискретизации, ≥ 10 мс;

target – желаемое расстояние до вагонетки, 0..450 мм;

servo – управляющее воздействие на сервопривод, 1500..4000.

Таблицу переменных *var_table[]* можно найти в файле «main.c» или вывести командой «help».

Работа программы

После запуска программы автоматы *usart1_send* и *usart1_recv* бездействуют, т.к. их состояния «*usart1_..._state*» равны «*USART1_..._NOT_ACTIVATED*». Первым запустившимся автоматом будет *picos*, который запустит *usart1_send*, для вывода приветственного сообщения, и *usart1_recv*, для получения команд пользователя. По умолчанию состояния автоматов *var_get* и *var_set* также равны «*..._NOT_ACTIVATED*», и они будут неактивны. Их работа регулируется автоматом *picos*, после получения соответствующих команд от пользователя.

Например, после получения автоматом *usart1_recv* команды «*led 1*» и её последующей обработки автоматом *picos*, он запустит автоматы *var_set*, который изменит значение переменной, и *var_get*, который и выведет изменённую переменную с её новым значением. Из-за изменения переменной *led*, сработает автомат *led_switch* и включит встроенный светодиод *PC13*.

На самом деле программа сложнее, т.к. обработка строки происходит за несколько циклов программы, светодиод включится раньше, чем пользователь получит строку с новым значением переменной и т.п.

Автомат *pid* включается не чаще, чем раз в 10 мс, после чего запускает автомат *distance_sensor* и после корректного измерения расстояния до вагонетки изменяет все переменные, необходимые для работы ПИД-регулятора и корректирует положение сервопривода. После завершения переходного процесса автомат включает светодиод 4.

Для работы с учебным стендом *BALUS* с возможностью изменения коэффициентов ПИД-регулятора:

1. Подключите *CH340* к учебному стенду (*GND* к *GND*, *TXD* к *RX*, *RXD* к *TX*);
2. Вставьте плату *CH340* в компьютер и подключитесь к новому *COM*-порту в программе *Terminal*;
3. Включите учебный стенд, подключив к нему блок питания или аккумулятор, после чего в *Terminal* будет выведено приветственное сообщение «Welcome to PicOS v3.4».

Для введения команд рекомендуется использовать окно посимвольного ввода в программе *Terminal* (самое нижнее). Все введённые команды требуют подтверждения – получение системой специального управляющего символа «+CR (Carriage Return)», который можно отправить нажатием на *Enter*, поэтому команду до подтверждения можно исправлять. Корректная работа программы и настройки *Terminal* изображены на рисунке 9.

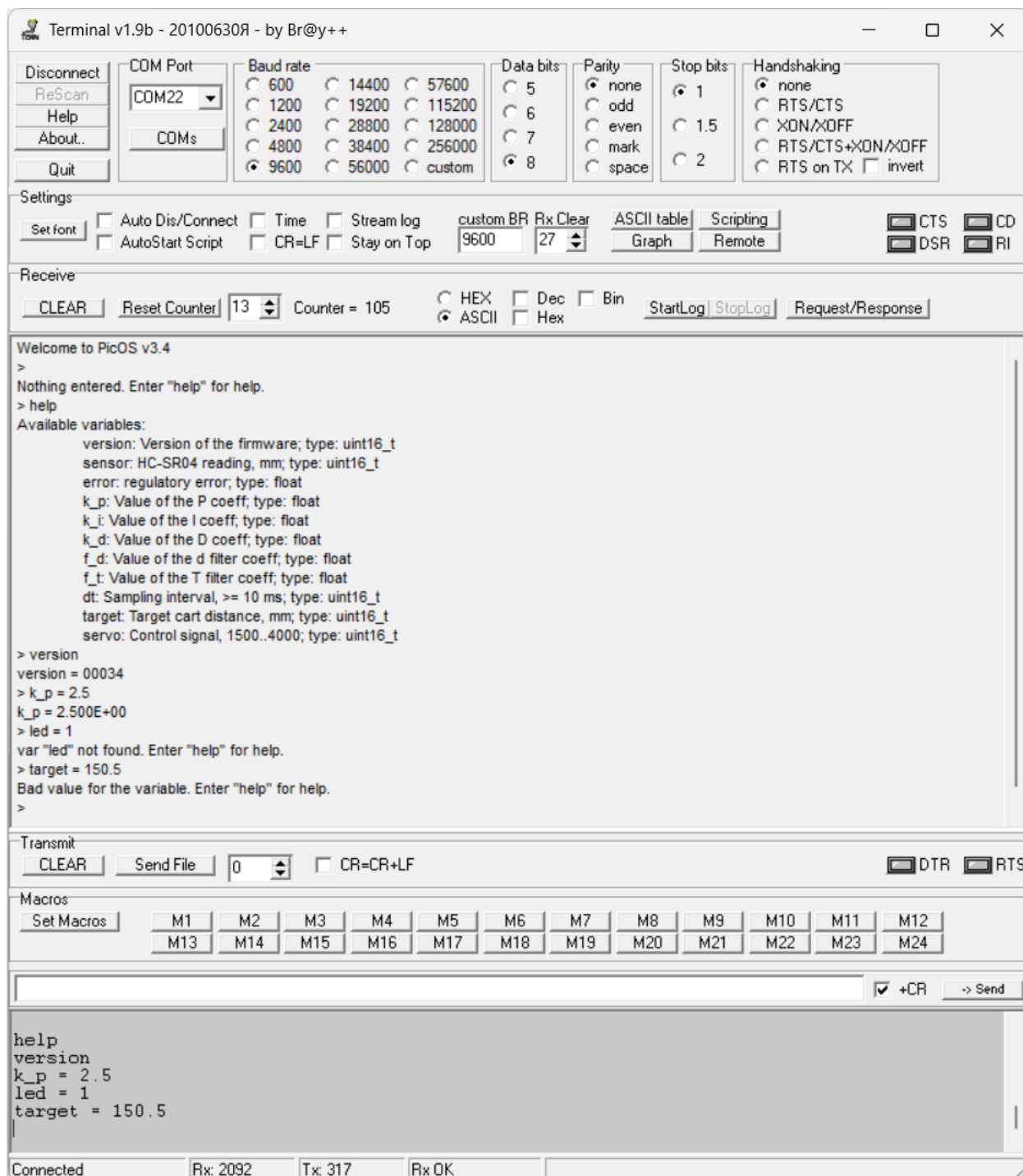


Рис. 9 Результаты работы программы BALUS-3 и настройки Terminal

Список литературы

1. Ультразвуковой датчик измерения расстояния HC-SR04 / В.А. Жмудь, Н.О. Кондратьев, К.А. Кузнецов, В.Г. Трубин, Л.В. Димитров // Автоматика и программная инженерия. 2017, №4(22). URL: <https://kb-au.ru/wp-content/uploads/AaSI-2017-4-2.pdf> (дата обращения: 13.10.2025).
2. Сервопривод / AmperMarket // URL: <https://ampermarket.kz/base/servomotors/> (дата обращения: 13.10.2025).
3. ПИД регулятор / alexgyver // URL: <https://alexgyver.ru/lessons/pid/> (дата обращения: 30.06.2025).
4. Использование дифференцирующего фильтра второго порядка для фильтрации сигналов акселерометра и определения производной / Д.С. Федоров, А.Ю. Ивойлов, В.А. Жмудь, В.Г. Трубин // Автоматика и программная инженерия. 2014, №4(10) // URL: <https://kb-au.ru/wp-content/uploads/AaSI-2014-4-10.pdf> (дата обращения: 30.06.2025).
5. Введение в автоматное программирование микроконтроллеров STM32 / В.А. Вдовин, И.В. Трубин, В.Г. Трубин, П.Д. Шендрик // Известия Томского политехнического университета. Промышленная кибернетика. –2024. –Т. 2. –No 4. –С. 21–25. DOI:10.18799/29495407/2024/4/75. // URL: <https://indcyb.ru/journal/article/view/75/60> (дата обращения: 01.02.2025).
6. Пример псевдопараллельного выполнения трёх задач на микроконтроллере STM32F103 без использования операционной системы / В.А. Вдовин, И.В. Трубин, В.Г. Трубин, П.Д. Шендрик // Известия Томского политехнического университета. Промышленная кибернетика. –2025. –Т. 3. –No 2. –С. 1–7. DOI:10.18799/29495407/2025/2/88 // URL: <https://indcyb.ru/journal/article/view/88/71> (дата обращения: 30.06.2025).